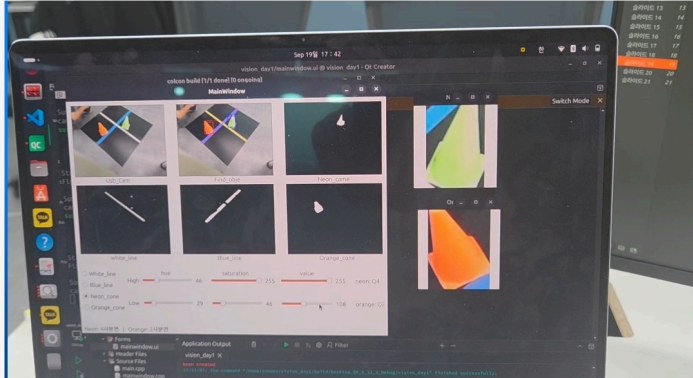


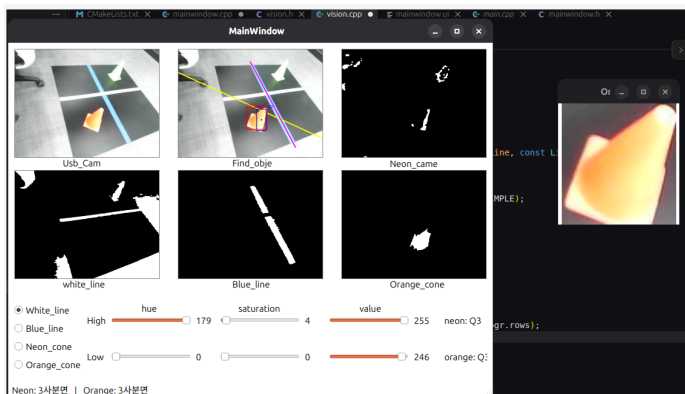
vision_day1_hw2 보고서

예비단원 김선우



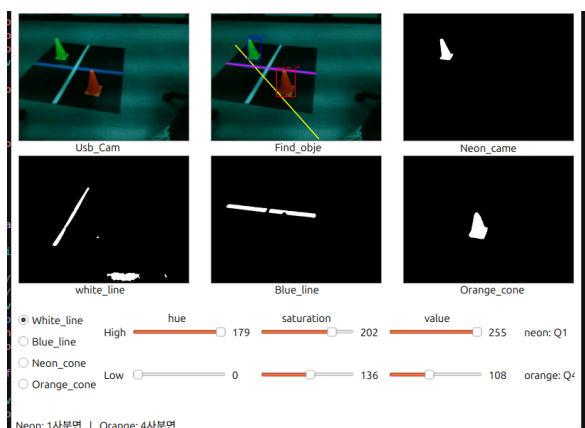
처음 실행했을 때의 화면이다. 해당 사진은 카메라를 위에 움직이지 못하도록 고정한 상태에서 콘의 위치를 옮기며 컴퓨터가 콘의 움직임을 따라가는지, 색을 잘 구분하는지를 테스트하였다. 사진을 보면 흰 선, 파란 선, 주황, 네온을 잘 구분하는 것을 볼 수 있다. 또한 콘의 위치를 바꿔도 잘 인식하는 것을 볼 수 있다.

하지만 문득 이걸 어디에 활용할까?라는 생각을 했을 때, 로봇이나 좀 있으면 하는 터틀봇 프로젝트에서 사용할 것 같았다. 그래서 카메라를 고정하지 않고 로봇처럼 움직임이 있도록 손으로 카메라를 흔들어보았다.



이 사진이 그 결과이다. 고정했을 때는 잘 인식하였지만, 조금 흔들면 바닥의 재질에 의한 빛반사로 일부 바닥이 흰색으로 인식되었다. 이로인해 흰 선이 잘 인식되지 않는 것을 볼 수 있다. 또한 네온색이 밝은 계열의 색이라 가끔씩 흰색으로도 인식되었고 어떨때는 흰 선과 네온 색이 구분이 안되는 경우도 많았다.

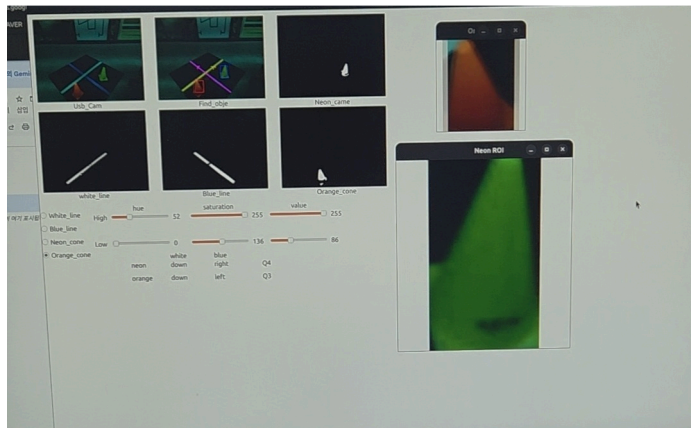
이 문제를 해결하기 위해 터미널을 통해서 자동 노출을 끄고 수동으로 전환하였고(auto_exposure=1), 셔터 시간을 6ms로 고정하여(exposure_time_absolute=60) 자동일 때에 비해 빛을 받는 시간을 줄였다. 또한 어두울 때 프레임을 낮추고 노출을 늘리는 기능(exposure_dynamic_framerate=0), 색감 자동 보정(white_balance_automatic=0), 역광 보정(backlight_compensation=0)을 켜었다.



이 사진이 그 결과이다. 네온, 주황 꼬깔을 보면 첫번째와 두번째 사진보다 더 윤곽이 잘 잡혀있고, 네모박스도 보다 정확하고, 꼬깔 이외의 다른 것들이 감지되지 않는 것을 볼 수 있다. 또한 카메라를 흔들었을 때도 파란선과 네온, 주황 꼬깔을 잘 인식하였다. 하지만 흰 선의 경우에는 아직 인식이 잘

되지 않았다. 또한 사분면 판단 기준이 없어 몇사분면에 위치해있는지 역시 알기 어려웠다.

영동한 흰 색을 선으로 인식하는 문제는 길쭉한 덩어리만 선 후보로 알아볼 수 있도록 ($kMinElongation = 2.5$)를 추가하였다. 또한 두 선의 교점을 원점으로 삼고, 흰 선을 x축, 파란 선을 y축으로 기준을 잡는다. 흰 선을 x축, 파란 선을 y축으로 보고 흰선 위/파란 선 오른쪽 -> 1Q(1사분면)이런 식으로 우리가 알아보기에 편하도록 수정하였다. 선의 굵기 안에 있으면 사분면이 아니라 on_line으로 판단한다. 추가로 x, y축이 잘 잡혔는지 확인하도록 화살표를 추가하였다.



이 사진이 최종 결과이다. 코드에 실험 환경에 맞게 흰색, 파란색, 네온, 오렌지를 잘 인식하는 범위를 박아냈다.

흰 선 인식을 잘 하고, 카메라에 진동이 가해져도 다른 빛에 흔들리지 않고 선이 꼬깔을 잘 잡아낸다.

프로그램이 콘을 찾는 과정.

가우시안 블러 -> BGR을 HSV로 변환 -> 마스크 만들기 -> 열기/닫기로 잡음 제거 -> 덩어리(윤곽선) 찾기 -> 바운딩박스 -> 위치 판단

1.가우시안 블러

```
// 1) 가우시안 블러로 노이즈 완화 -> HSV 변환
cv::Mat blurred, hsv;
cv::GaussianBlur(bgr, blurred, cv::Size(5, 5), 0);
cv::cvtColor(blurred, hsv, cv::COLOR_BGR2HSV);
```

영상처리를 할 때 어떤 픽셀이 주변 픽셀과 극단적으로 다른 값을 가지면 노이즈가 생기는데, 이미지가 선명할수록 물체 사이에 날카로운 윤곽선이 생기기 때문이다. 그래서 색을 나누기 전에 가우시안 블러를 먼저 적용하여 튀는 값을 완화하였다. 이미 과제 1에서 다뤘지만, 가우시안 필터는 창 안의 픽셀을 평균 내지만, 중심 픽셀에 가까운 이웃일수록 큰 가중치를 주는 필터이다. 예를 들어 3×3 커널은 가운데가 4, 상하좌우가 2, 대각선이 1이고 합인 16으로 나눈다. 그래서 모든 픽셀에 같은 비중을 주는 평균 필터보다 원래 픽셀의 정보를 더 남기면서 부드럽게 흐려진다. 코드에서는 5×5 커널을 사용하였고, σ 를 0으로 주면 OpenCV가 커널 크기에 맞춰 알아서 정한다.

2. BGR을 HSV로 변환

OpenCV는 이미지를 RGB가 아니라 BGR 순서로 읽는다. RGB는 세 값이 모두 색과 밝기를 함께 나타내기 때문에 "이 색은 이 범위"라고 지정하기가 어렵다. 반면 HSV는 색조(Hue), 채도(Saturation), 명도(Value)로 나누어 표현하기 때문에 색은 H로, 밝기는 V로 따로 다룰 수 있어서 색을 구분하기 편하다. OpenCV에서 H는 0~179, S와 V는 0~255의 값을 가진다. 이는 ui를 통해 사용자가 환경에 맞게 조절할 수 있도록 하였다.

3. 마스크 만들기

각 픽셀의 H, S, V가 모두 하한과 상한 사이에 있으면 흰색(255), 하나라도 벗어나면 검은색(0)으로 바꿔서 이진(바이너리) 이미지로 만드는 함수이다. 이렇게 만들어진 이미지를 마스크라고 부르고, 흰 선, 파란 선, 네온 콘, 주황 콘 4가지 색에 대해 각각 마스크를 만들었다. 마스크를 통해 어떤색이 인식이 잘 되는지 안되는지를 판단하고 H, S, V값을 수정하여 보다 좋은 결과를 만들 수 있게 하였다.

4. 열기/닫기로 잡음 제거

열기는 침식을 먼저 한 뒤 팽창을 하는 연산으로, 작은 점 잡음을 지우는 데 효과적이다. 닫기는 팽창을 먼저 한 뒤 침식을 하는 연산으로, 물체 안쪽의 구멍이나 작은 틈을 메우는 데 효과적이다. 횡수를 늘리면 잡음은 더 지워지지만 가는 선까지 깎아서 흰 선과 파란 선이 사라질 수 있어 각각 한 번씩만 적용하였다.

5. 덩어리 찾기

덩어리는 앞선 과제1에서 한 번 사용했었던 것을 같은 개념으로 거의 똑같이 사용하였다. 똑같이 양 옆, 위/아래, 대각선을 포함한 8개의 픽셀을 붙어있는 것으로 간주하였다. 면적이 200픽셀 미만인 덩어리는 잡음으로 보고 버렸다. 긴 축과 짧은 축의 비율이 2.5 이상인 길쭉한 덩어리만 선 후보로 인정하였다(kMinElongation = 2.5). 바닥의 빛반사처럼 뭉뚱한 덩어리를 걸러내기 위해서이다. 흰 선이 가로로 놓일 수도, 세로로 놓일 수도 있기 때문에 방향을 고정하지 않았다. 대신 파란 선을 먼저 찾고, 흰 선은 파란 선과 45° 이상 벌어진(수직에 가까운) 덩어리만 후보로 인정하였다. 흰 선은 하나뿐이므로 가장 큰 후보 하나를 선택하였고, 파란 선은 흰 선에 의해 여러 조각으로 끊길 수 있어서 가장 큰 덩어리 넓이의 20% 이상인 조각을 모두 선택하였다.

콘의 경우에는 findContours로 윤곽선을 찾고, contourArea로 넓이를 재서 가장 큰 덩어리를 콘으로 보았다. 넓이가 300픽셀보다 작으면 잡음으로 판단하여 콘이 없다고 처리하였다.

6. 바운딩박스

```

r.found = true;
r.box = cv::boundingRect(contours[idx]) & cv::Rect(0, 0, bgr.cols, bgr.rows);
r.center = (r.box.tl() + r.box.br()) / 2;
r.pos = classify(r.center, ax, white, blue, r.whiteSide, r.blueSide);
r.roi = bgr(r.box).clone(); // 바운딩박스 내부만 추출

```

콘의 윤곽선을 감싸는 가장 작은 사각형을 **boundingRect**로 구하여 바운딩박스로 사용하였다. 박스의 중심점은 왼쪽 위와 오른쪽 아래 좌표의 가운데로 잡았고, 이 중심점이 위치 판단의 기준이 된다. 박스가 화면 밖으로 나가지 않도록 화면 영역과 겹치는 부분만 남겼다. 과제 조건에 따라 콘이 인식되면 박스 안쪽만 잘라서 새 이미지(ROI)로 만들고 별도의 창에 띄웠다. 화면에는 네온 콘은 파란 박스, 주황 콘은 빨간 박스로 그리고 위치 결과를 글자로 함께 표시하였다. (바운딩박스는 축에 나란한 사각형이므로 만약 콘의 각도가 눕혀있다면, 박스가 커진다.)

7. 위치판단

두 선의 교점을 원점으로 삼고 흰 선을 **x**축, 파란 선을 **y**축으로 보았다. 콘의 중심점이 흰 선의 위쪽이면 **up**, 아래쪽이면 **down**, 파란 선의 오른쪽이면 **right**, 왼쪽이면 **left**로 판단하고, 이 두 값의 조합으로 사분면을 정하였다. 콘의 중심점에서 선까지의 수직 거리가 선 굵기의 절반에 여유 **5**픽셀을 더한 값 이하이면 그 선 위에 있다고 보고 사분면 대신 **on line**으로 표시하였다. 흰 선 위이면 **White-Right/Left**, 파란 선 위이면 **Blue-Up/Down**으로 나타냈고, 두 선이 만나는 곳은 **Center**이다. 콘이 없으면 **None**, 기준선이 검출되지 않으면 판별 불가(**Unknown**)로 처리하였다. 두 선이 정확히 **90°**가 아니어도(카메라가 비스듬할 때) 사분면이 맞게 나오도록, 콘의 위치를 두 선의 방향을 축으로 하는 기울어진 좌표계에서 계산하였다. 또한 좌표계가 맞게 잡혔는지 눈으로 확인할 수 있도록 원점에서 **+x, +y** 방향 화살표를 화면에 그렸다.

콘 위치 판단 코드에 쓰인 함수 리스트 & **vision** 개념

입력

-VideoCapture (cap.open, cap.set, cap.read)카메라 열기, 해상도와 형식 설정, 프레임 읽기

전처리

-GaussianBlur5×5 가우시안 필터로 노이즈 완화cvtColorBGR을 HSV로 변환 (Qt 화면 표시용으로 BGR을 RGB로도 변환)

색 분리

-inRangeHSV 범위 안이면 흰색, 밖이면 검은색인 이진 마스크 생성

잡음 정리

-getStructuringElement5×5타원형 커널 생성

-morphologyEx열기(MORPH_OPEN)로 작은 점 제거, 닫기(MORPH_CLOSE)로 구멍 메우기

선 찾기

- connectedComponentsWithStats 붙어 있는 흰 픽셀을 덩어리로 묶고 면적과 위치 정보 얻기
- moments 덩어리의 길쭉한 정도와 방향(각도) 계산
- countNonZero 흰 픽셀 수(면적) 세기
- findNonZero 흰 픽셀들의 좌표 목록 얻기
- fitLine 그 좌표들에 가장 잘 맞는 직선 구하기

콘 찾기

- findContours_마스크에서 윤곽선 찾기
- contourArea 윤곽선 넓이 재서 가장 큰 덩어리를 콘으로 선택
- boundingRect 콘을 감싸는 바운딩박스 구하기

그리기

- line 인식한 흰 선(노란색), 파란 선(분홍색) 그리기
- arrowedLine 좌표계의 x, y 방향 화살표 그리기
- rectangle 바운딩박스 그리기
- circle 박스 중심점 그리기
- putText 박스 위에 위치 결과 글자 쓰기

모폴로지란? 이미지에 생기는 노이즈를 제거하여 이미지를 보다 자연스럽게/매끄럽게하는 작업

모폴로지 - 침식

객체 영역의 바깥부분을 깎는 연산. 이 때 대각선을 제외한 양 옆/위아래 부분의 픽셀을 탐색하고 해당 위치에 픽셀이 없을 시 픽셀 삭제.

모폴로지 - 팽창

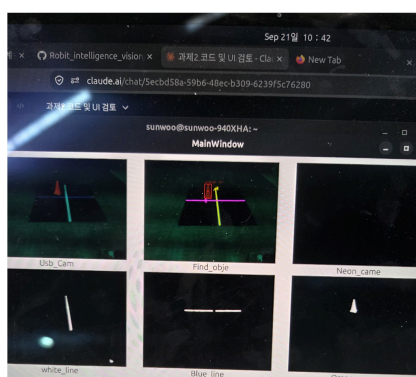
객체 영역을 확장하는 연산. 이 때 커널 범위 안에 픽셀이 하나라도 있으면 그 위치까지 픽셀을 채운다. 그 결과 객체가 커지고, 객체 안쪽의 작은 구멍이나 끊어진 틈이 메워진다.

모폴로지 - 열기

침식을 먼저 한 후, 다시 팽창을 수행하는 연산. 침식으로 작은 점 잡음을 먼저 없애고, 팽창으로 깎여서 작아진 객체를 원래 크기로 되돌린다. 그래서 객체의 크기는 유지하면서 잡음만 제거할 때 효과적이다.

모폴로지 - 닫기

팽창을 먼저 한 후, 다시 침식을 수행하는 연산. 팽창으로 객체 안쪽의 구멍이나 틈을 먼저 메우고, 침식으로 커진 객체를 원래 크기로 되돌린다. 그래서 객체中间的 빈 구멍을 없앨 때 효과적이다.



동영상에 꼬깔이 화면 밖으로 나갔을 때가 없어서
보고서에 따로 첨부하겠습니다. 네온 꼬깔만 화면 밖으로
이동시킨 모습입니다. 검은 매트의 범위를 파악하지

못해서 매트가 아닌 맨바닥에 꼬깔이 있어도 몇 사분면인지 위치가 뜹니다.